# RAG Study Lab System Architecture

Monorepo, retrieval pipeline, security boundaries, and deployment profile

This document describes the current architecture of RAG Study Lab. It is a study-focused implementation that keeps the same important boundaries found in a production RAG service: explicit retrieval, tenant isolation, private ingestion, provider adapters, evaluation, and cost-aware deployment.

## System overview

| Layer | Responsibility | Current implementation |
|---|---|---|
| Web | Interactive RAG lab and protected administration | React, TypeScript, Tailwind CSS, TanStack Query, Feature-Sliced Design |
| API | Authorization, orchestration, ingestion, retrieval, Q&A | Fastify, Clean Architecture, LangGraph |
| Data | Tenant-scoped documents, chunks, jobs, audit records | PostgreSQL, pgvector, full-text search |
| Identity and files | Sessions and private original documents | Supabase Auth and private Storage |
| AI providers | Embeddings, answer generation, optional reranking | Ollama, OpenRouter, Bedrock, optional Jina |

## Request and data flow

| Step | Flow |
|---|---|
| 1 | The browser uses a Supabase session token. It never receives provider, database, service-role, or administrative secrets. |
| 2 | The Fastify API validates the session, resolves the caller membership, and derives the allowed workspace. |
| 3 | For ingestion, an upload becomes a durable job. Text is extracted, chunked, scanned, embedded, and stored transactionally. |
| 4 | For questions, hybrid retrieval finds context, safety checks remove risky chunks, and LangGraph produces a grounded response with sources. |
| 5 | The frontend renders the answer, retrieval sources, document management state, and an ingestion progress visualization. |

## Monorepo structure

The repository uses npm workspaces. Clear ownership prevents browser concerns, API concerns, and shared validation from becoming interleaved.

| Path | Ownership |
|---|---|
| apps/web | Feature-sliced React frontend, public configuration, and authenticated API client |
| apps/api | HTTP and CLI interfaces, use cases, adapters, migrations, workers, and evaluations |

| | |
|---|---|
| packages/contracts | Zod schemas and inferred types shared by the API and the web application |
| docs | Architecture, deployment, and product backlog |
| examples | Versioned learning corpus used by local evaluation |

## API architecture

The API follows Clean Architecture. Dependencies point inward, and concrete integrations are selected in bootstrap.ts.

| Boundary | Role | Examples |
|---|---|---|
| Domain | Rules with no framework dependencies | Documents, chunks, prompt-injection policy |
| Application | Use cases and ports | Ingest, search, answer, queue upload, member management |
| Infrastructure | Concrete adapters | pgvector, Supabase Storage, Ollama, OpenRouter, Bedrock, Jina |
| Interfaces | Transport translation | Fastify routes and CLI commands |
| Bootstrap | Composition root | Provider selection and dependency wiring |

## Ingestion pipeline

| Stage | Purpose | Safety and quality consideration |
|---|---|---|
| Upload | Accept an authenticated PDF, DOCX, TXT, or Markdown file | Role check, MIME and size restrictions |
| Extract | Convert supported source into text | Scanned PDFs require future OCR |
| Chunk | Create bounded retrieval units with overlap | Chunk size balances recall, context, and cost |
| Scan | Classify injection-like instructions | High-risk chunks are excluded from generated answers |
| Embed | Map safe chunks to vectors | Provider stays server-side; model dimension must remain consistent |
| Persist | Replace tenant-scoped chunks and update job | Transaction prevents partial replacement |

## Hybrid retrieval and grounded answers

A question is evaluated through two retrieval channels. Vector search handles semantic similarity, while full-text search handles exact terminology. Reciprocal Rank Fusion merges their positions without pretending that their raw scores are directly comparable.

| Phase | Operation | Output |
|---|---|---|
| Retrieve | Query embedding + pgvector cosine similarity; PostgreSQL full-text search | Candidate chunks with rank signals |
| Fuse | Reciprocal Rank Fusion | Single candidate ordering |
| Filter | Remove high-risk chunks and enforce tenant scope | Safe candidate context |

| | | |
|---|---|---|
| Rerank | Optional specialized reranker over a limited candidate set | Higher-quality top results |
| Answer | LangGraph retrieve → prepare context → generate answer | Validated answer and source citations |

Retrieval score is a relative relevance indicator, not factual confidence. The user interface exposes sources because the answer must remain inspectable.

## Identity, tenancy, and authorization

| Role | Allowed actions |
|---|---|
| Owner | Create workspace, manage members, invite users, and manage the corpus |
| Editor | Upload, reindex, retry, and delete documents in the selected workspace |
| Viewer | Inspect documents and chunks without modifying the corpus |

The API validates the Supabase bearer token on every protected route and resolves membership from the database. The workspace header is a selection hint only; it is accepted only when it belongs to the authenticated caller.

## Security controls

| Threat | Control |
|---|---|
| Prompt injection in documents | Scanner, untrusted-context delimiters, high-risk chunk exclusion, regression suite |
| Cross-workspace access | Mandatory tenant filters, membership-derived authorization, role checks |
| Browser secret leakage | Server-only provider, database, service-role, and administrative keys |
| Cost abuse | Daily tenant query and administrator upload quotas, compact audit trail |
| Untrusted model output | Zod validation and server-side source validation |

## Quality and operational roadmap

| Now | Next |
|---|---|
| Unit tests, golden set, injection suite, Promptfoo gate | Run every evaluation gate in CI before deployment |
| Tenant quotas and audit metadata | PII redaction, per-user quotas, semantic cache, usage and budget alerts |
| Private async ingestion | OCR, URL/HTML import, language detection, structure-aware chunking |
| Render + Supabase demonstration deployment | Dedicated worker, backups, retention policy, RLS before direct data access |

## Low-cost deployment profile

For demonstrations, the frontend is a Render static site, the API is a Render web service, and Supabase provides Postgres, pgvector, authentication, and private Storage. OpenRouter supplies hosted AI only when local Ollama is not appropriate. This design keeps the browser static, moves secrets to the API environment, and avoids dedicated infrastructure until workload justifies it.

| Component | Responsibility | Cost-aware default |
| --- | --- | --- |
| Static web app | Serve the React UI globally | Render static site with `apps/web/dist` |
| API | Keep authorization and provider keys server-side | Single Render web service built from `apps/api/Dockerfile` |
| Data platform | Postgres, Auth, private originals | Supabase project with private Storage |
| AI | Embeddings and grounded answers | Local Ollama for study; OpenRouter only when hosted inference is needed |

Deployment acceptance criteria include a healthy API, a browser bundle free of secrets, restricted CORS, successful type checks and tests, and passing evaluation gates. Before onboarding real users, add PII redaction, usage alerts, backups, retention rules, and a security review.